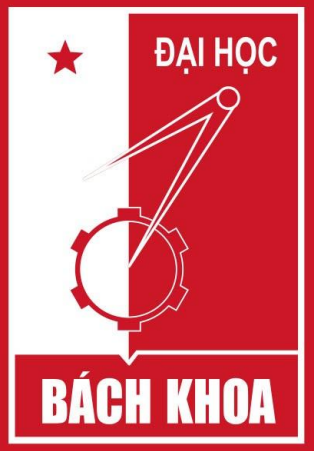




ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

Lecture 5
Architecture

Content

1. Overview
2. Architecture characteristics
3. Architecture styles
4. Documenting the architecture

1. Overview

What is software architecture?



Problems

- Requirements are subject to change
  - We need a stable model to minimize the cost of changing.
- System model in details is extremely complex
  - We need a high-level model to represent the system for the sake of communication.

Definition

"Software architecture is the high-level organization of a system. It defines the core components, how they interact, and the design rules that guide future growth."

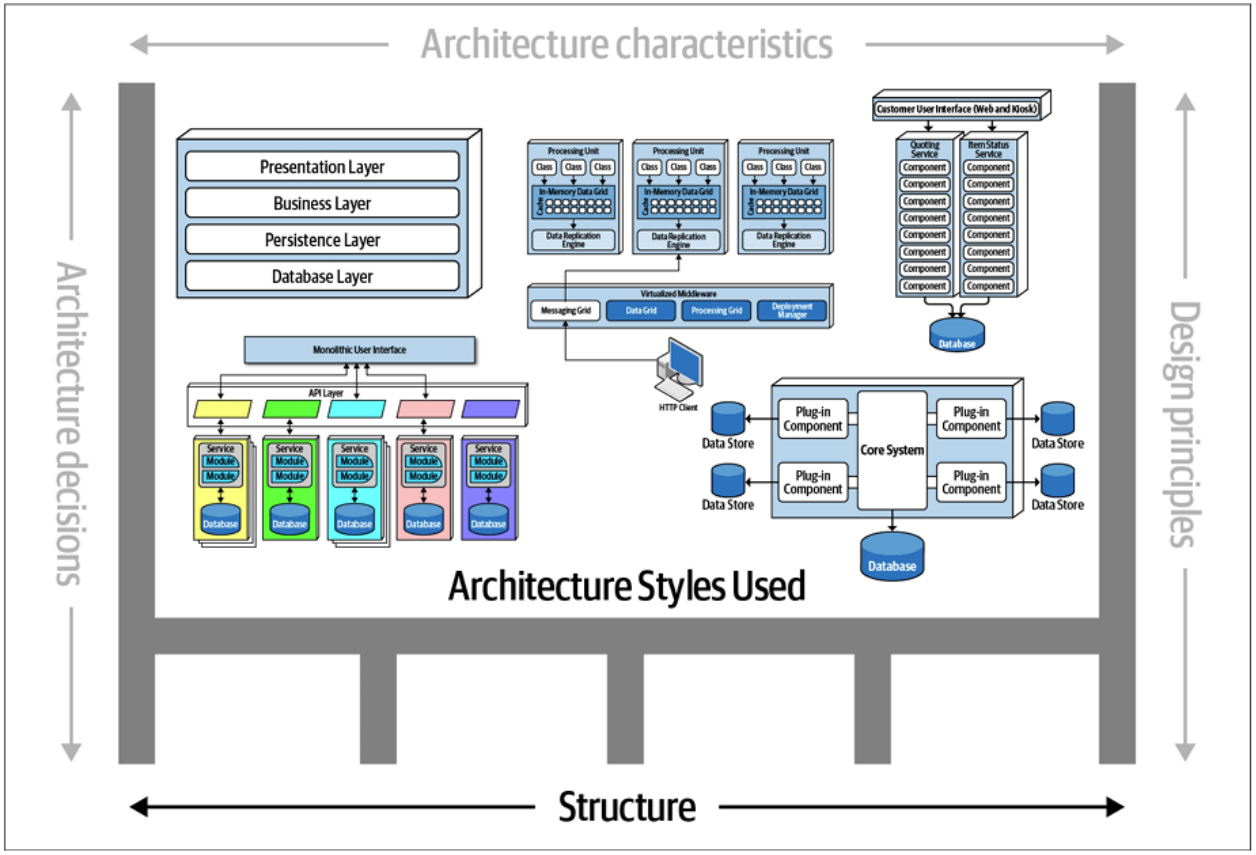
IEEE Computer Society

- Two fundamental laws in software architecture (Neil Ford):
  - "Everything is a tradeoff"
  - "Why is more important than how"

Dimensions of Software Architecture

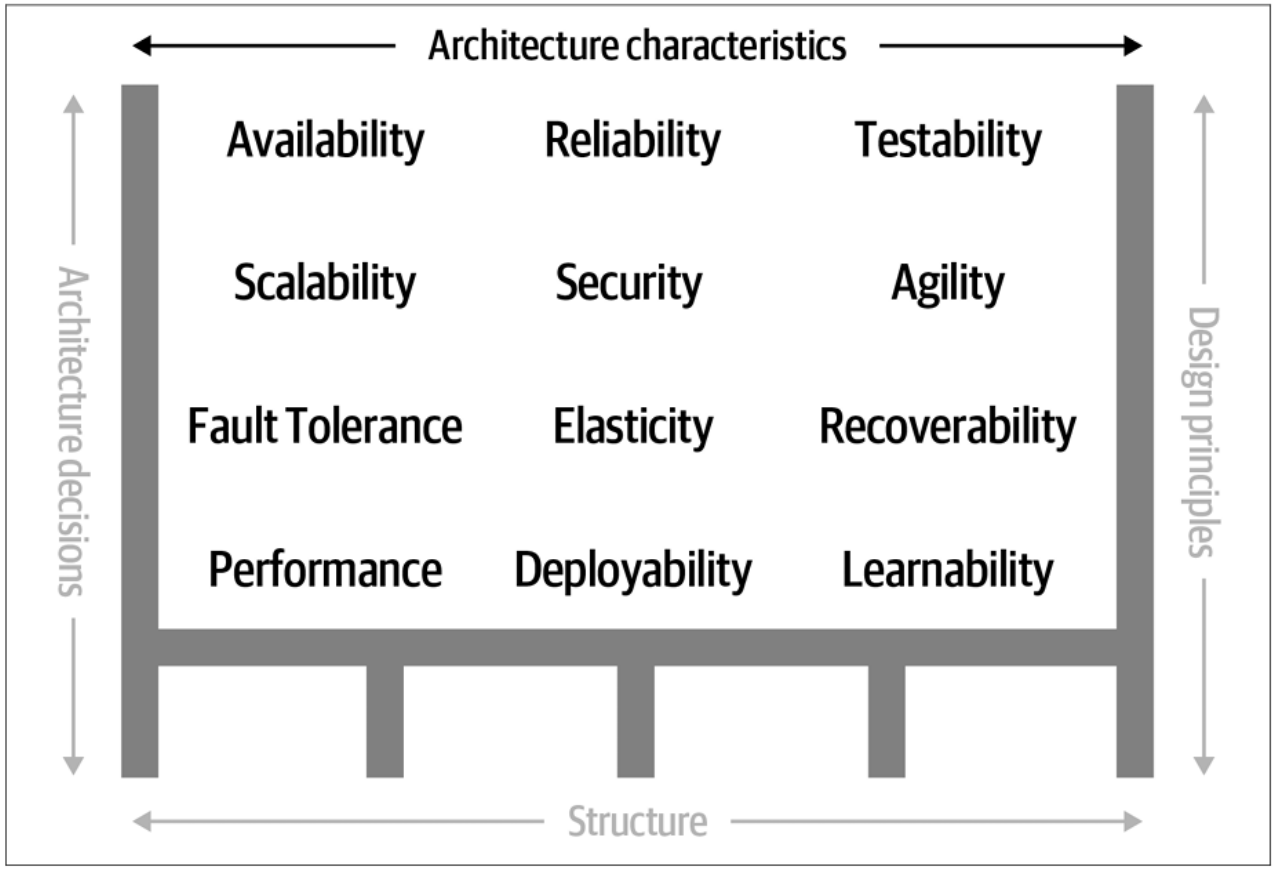
- Software architecture consists of (by Neil Ford)
  - Structure
  - Architecture characteristics
  - Architecture decisions
  - Design principles

Structure



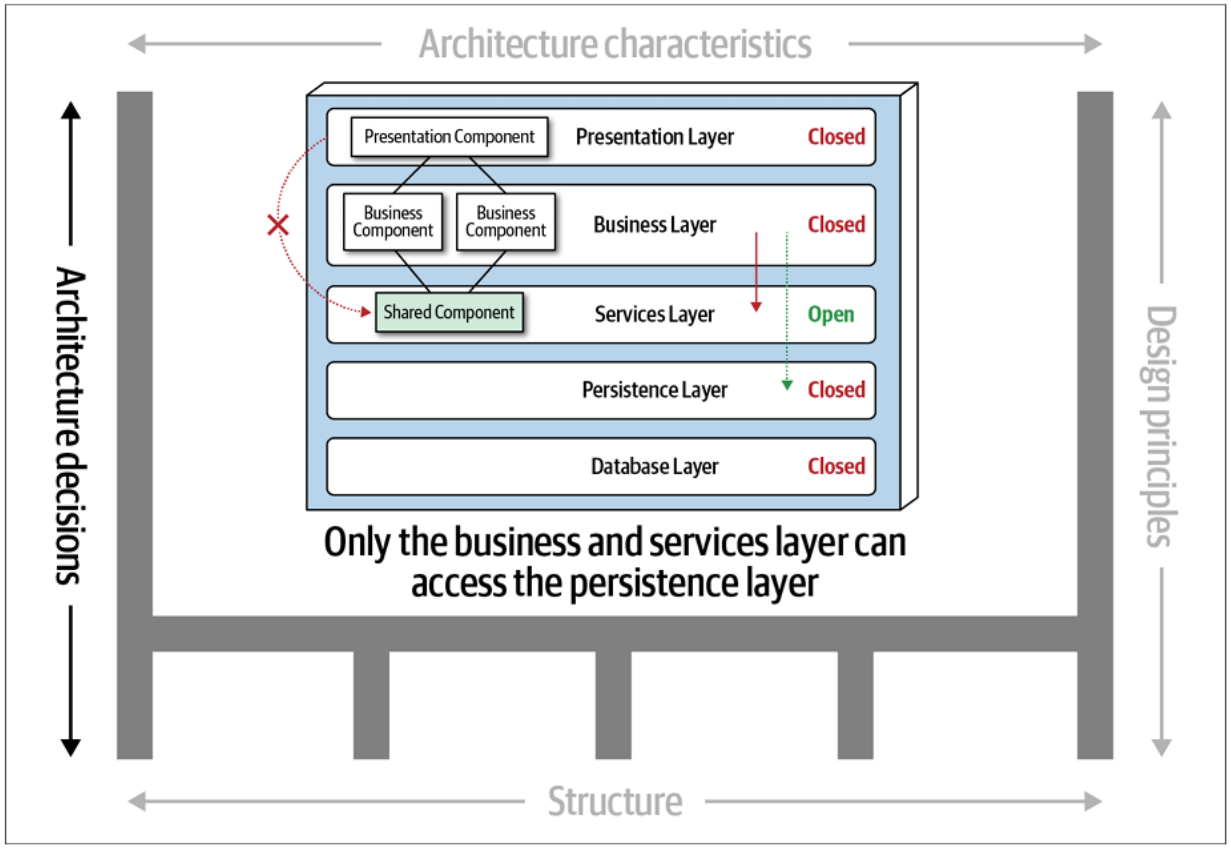
Structure normally refers to the type of architecture styles used in the system

Architecture Characteristics



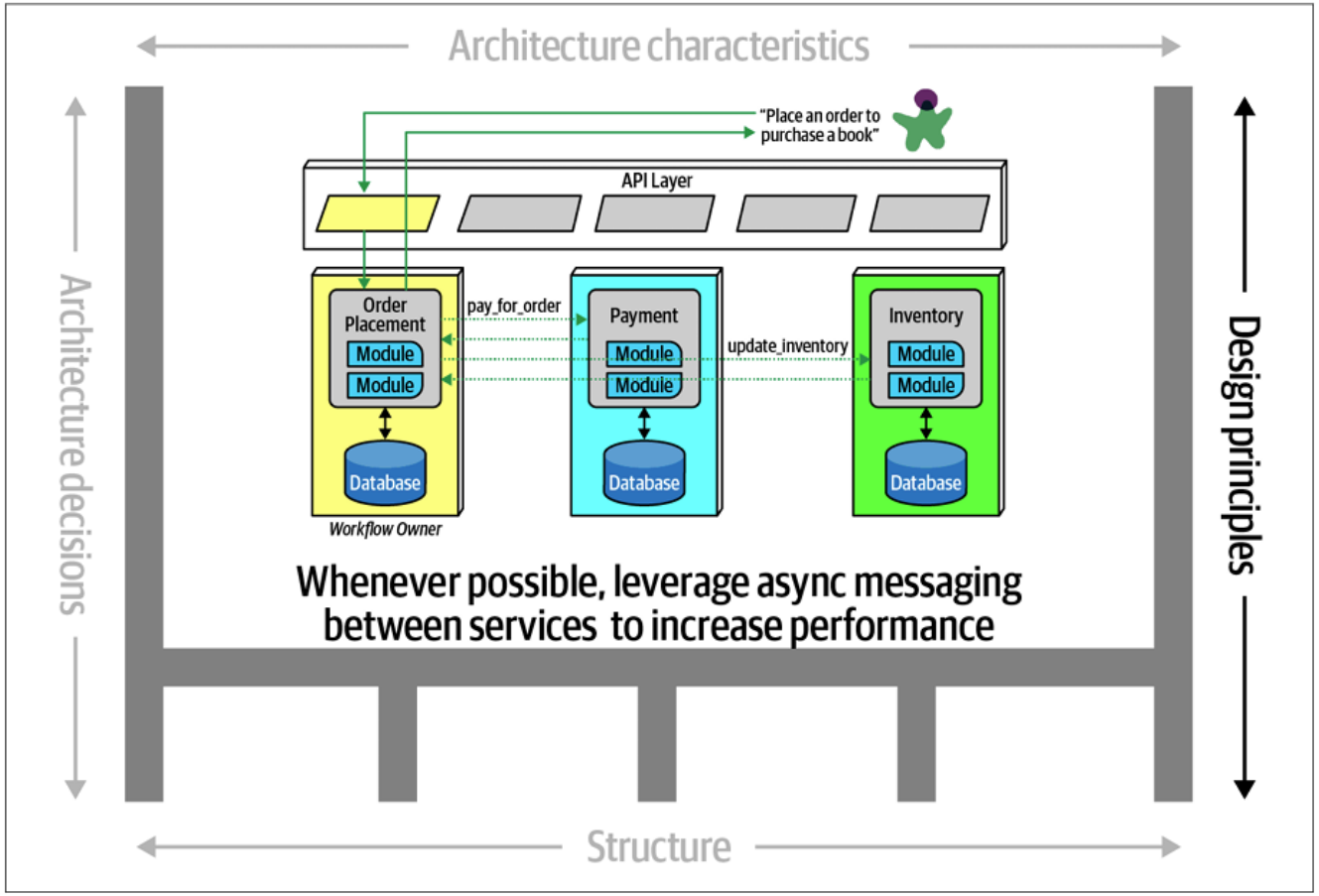
Architecture characteristics refers to the "-ilities" that the system must support

Architecture Decisions



Architecture decisions are rules for constructing systems

Design Principles



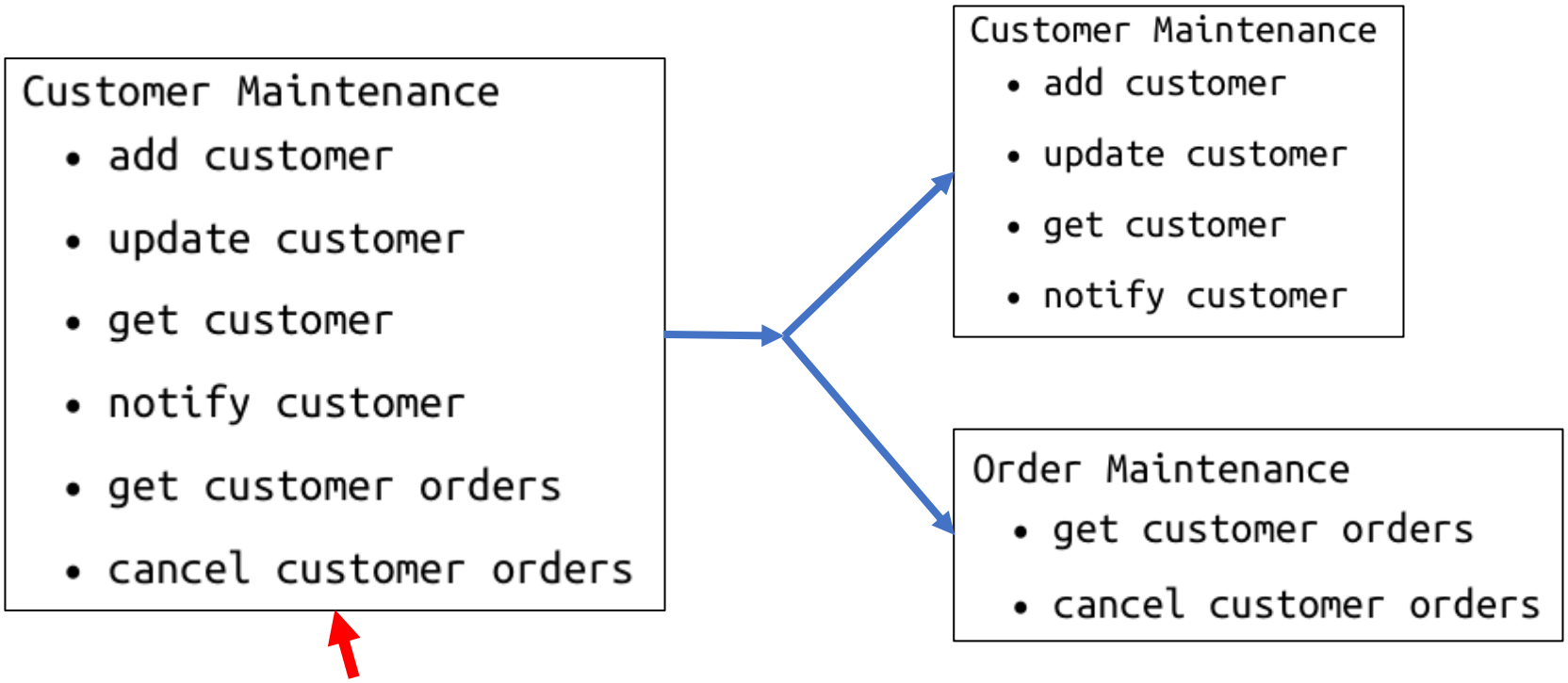
Design principles are guidelines for constructing systems

Modularity

- Creating an architecture is a process of decomposing a system into small pieces, regrouping them into modules (normally in hierarchical manner), and specifying the relationship among them.
- Modularity refers to measurements of how good small pieces of the system are group together into modules:
  - Cohesion
  - Coupling

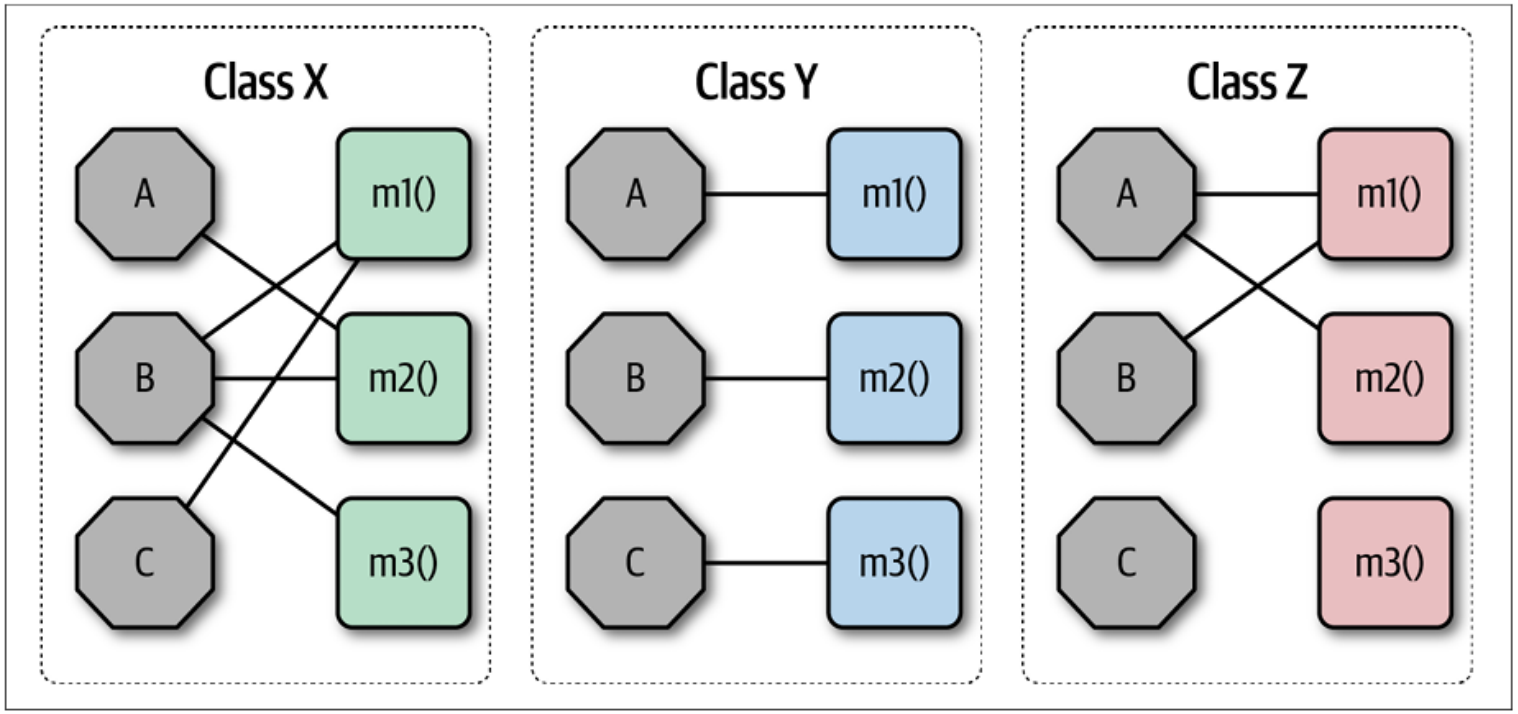
Cohesion

- Cohesion refers to what extent the parts of a module should be contained within the same module.
  - (Ideal) Cohesive module
    - all the parts should be packaged together;
    - breaking them into smaller modules would require coupling the parts together via calls between modules.



Cohesion - LCOM

- LCOM : Lack of Cohesion in Methods



LOW (Good) HIGH (Bad) MED (Avg)

NOT A COHESIVE MODULE

Coupling

- Coupling defines how much different modules depend on each other.
  - Tightly couple: change in one module forces changes in the others
  - Loosely couple: change in one module doesn't affect the others
- Metrics:
  - Afferent coupling: number of incoming connection
  - Efferent coupling: number of outgoing connection
  - Abstractness:

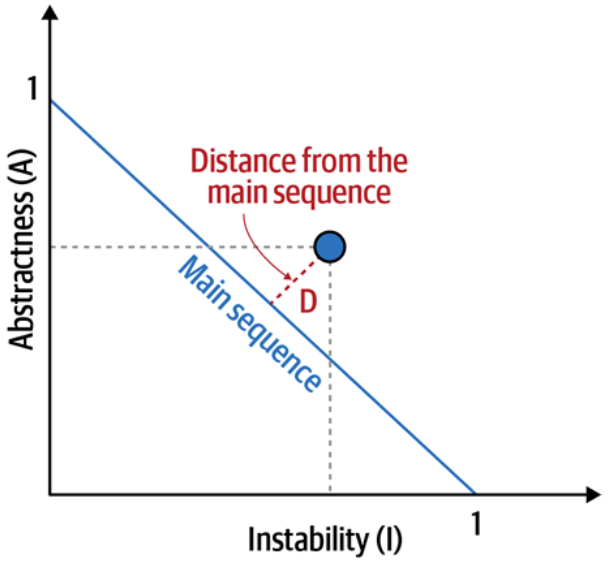
$$A = \frac{\sum m^a}{\sum m^c}$$

  - Instability:

$$I = \frac{C^e}{C^e + C^a}$$

  - Distance from main sequence:

$$D = |A + I - 1|$$



NOT A COHESIVE MODULE

2. Architecture Characteristics

Architecture Characteristics

Architecture Characteristics = Non-Functional Requirements

- Architecture characteristics meet:
  - Specifies a nondomain design consideration
  - Influences some structural aspect of the design
  - Is critical or important to application success
- Categories:
  - Operational architecture characteristics
  - Structural Architecture Characteristics
  - Cross-cutting architecture characteristics

Operational Architecture Characteristics

Term	Definition
Availability	How long the system will need to be available (if 24/7, steps need to be in place to allow the system to be up and running quickly in case of any failure).
Continuity	Disaster recovery capability.
Performance	Includes stress testing, peak analysis, analysis of the frequency of functions used, capacity required, and response times. Performance acceptance sometimes requires an exercise of its own, taking months to complete.
Recoverability	Business continuity requirements (e.g., in case of a disaster, how quickly is the system required to be on-line again?). This will affect the backup strategy and requirements for duplicated hardware.
Reliability/ safety	Assess if the system needs to be fail-safe, or if it is mission critical in a way that affects lives. If it fails, will it cost the company large sums of money?
Robustness	Ability to handle error and boundary conditions while running if the internet connection goes down or if there's a power outage or hardware failure.
Scalability	Ability for the system to perform and operate as the number of users or requests increases.

Structural Architecture Characteristics

Term	Definition
Configurability	Ability for the end users to easily change aspects of the software's configuration (through usable interfaces).
Extensibility	How important it is to plug new pieces of functionality in.
Installability	Ease of system installation on all necessary platforms.
Leverageability/ reuse	Ability to leverage common components across multiple products.
Localization	Support for multiple languages on entry/query screens in data fields; on reports, multibyte character requirements and units of measure or currencies.
Maintainability	How easy it is to apply changes and enhance the system?
Portability	Does the system need to run on more than one platform? (For example, does the frontend need to run against Oracle as well as SAP DB?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Upgradeability	Ability to easily/quickly upgrade from a previous version of this application/solution to a newer version on servers and clients.

Cross-cutting Architecture Characteristics

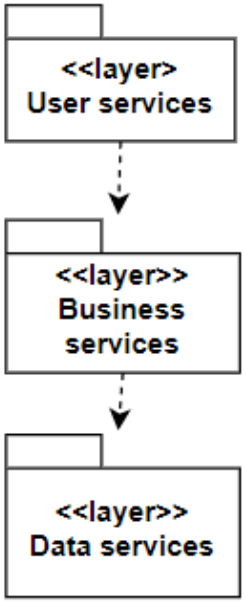
Term	Definition
Accessibility	Access to all your users, including those with disabilities like colorblindness or hearing loss.
Archivability	Will the data need to be archived or deleted after a period of time? (For example, customer accounts are to be deleted after three months or marked as obsolete and archived to a secondary database for future access.)
Authentication	Security requirements to ensure users are who they say they are.
Authorization	Security requirements to ensure users can access only certain functions within the application (by use case, subsystem, webpage, business rule, field level, etc.).
Legal	What legislative constraints is the system operating in (data protection, Sarbanes Oxley, GDPR, etc.)? What reservation rights does the company require? Any regulations regarding the way the application is to be built or deployed?
Privacy	Ability to hide transactions from internal company employees (encrypted transactions so even DBAs and network architects cannot see them).
Security	Does the data need to be encrypted in the database? Encrypted for network communication between internal systems? What type of authentication needs to be in place for remote user access?
Supportability	What level of technical support is needed by the application? What level of logging and other facilities are required to debug errors in the system?
Usability/ achievability	Level of training required for users to achieve their goals with the application/solution. Usability requirements need to be treated as seriously as any other architectural issue.

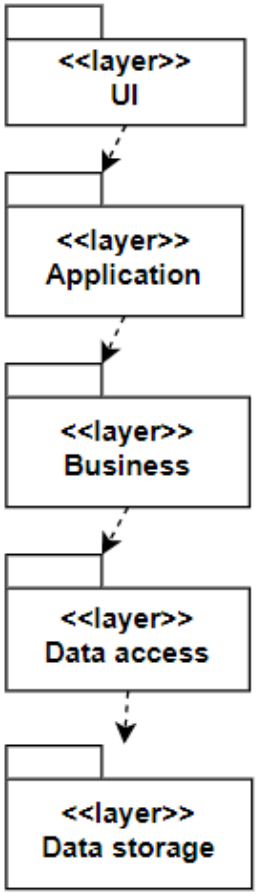
3. Architecture Styles

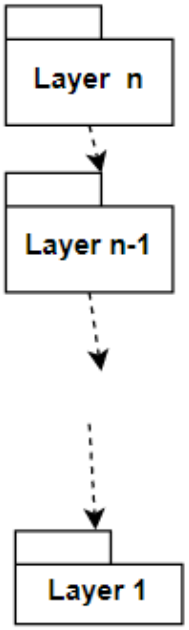
Architectural Styles

- Mẫu kiến trúc phần mềm là một giải pháp chung, có thể tái sử dụng cho một tình huống thường xảy ra trong một ngữ cảnh nào đó
- Phong cách kiến trúc – architectural styles
- Một số mẫu kiến trúc phổ biến:
  - Monolithic
    - Big Ball of Mud
    - Layered
    - Microkernel
  - Distributed
    - Pipes & Filters
    - Microservices
    - ESB-SOA
    - Serverless

Mẫu kiến trúc phân tầng (layered)







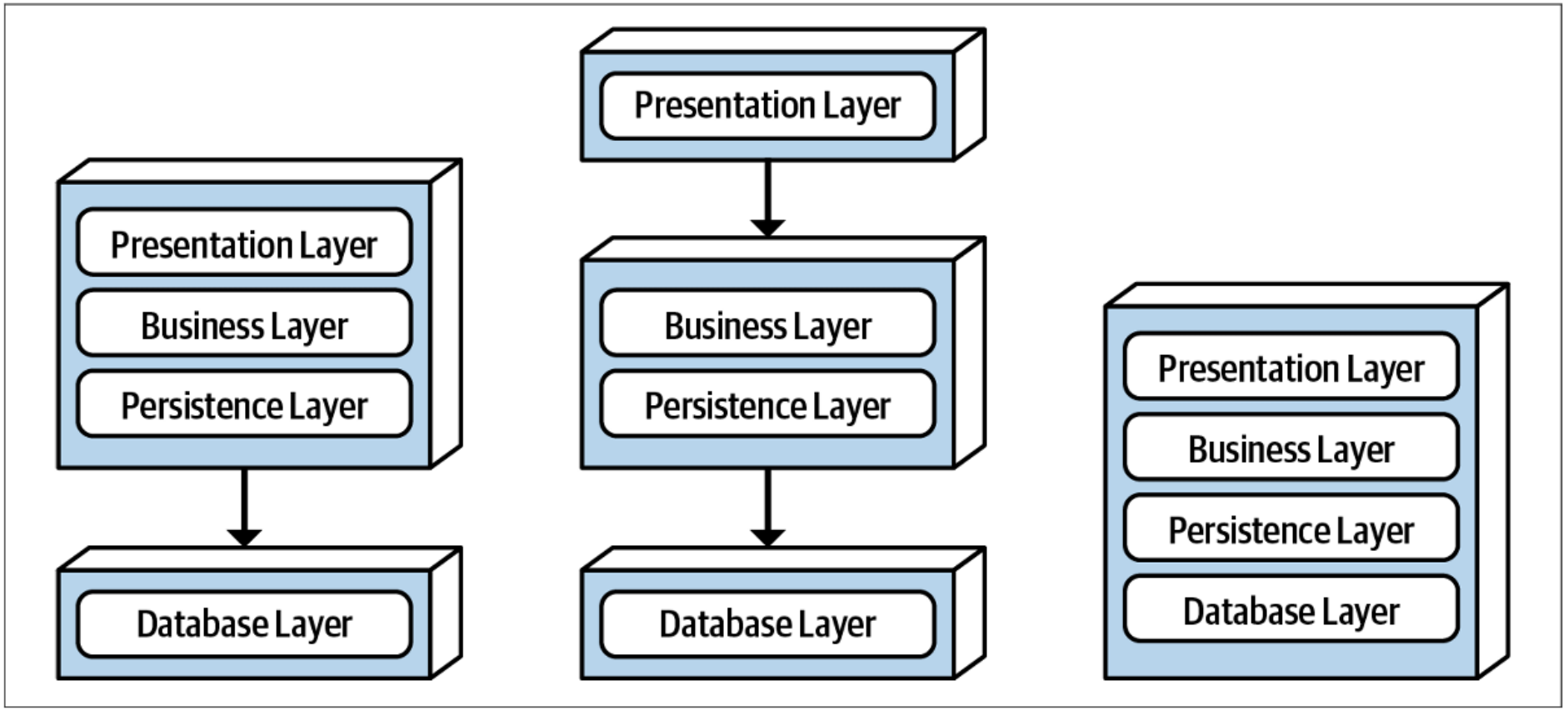
Mẫu kiến trúc phân tầng

- Các lớp của hệ thống được gom nhóm vào n tầng, tầng trên phụ thuộc tầng dưới
  - Không được phép phụ thuộc "ngược" – tầng dưới phụ thuộc tầng trên
  - Hạn chế thiết lập phụ thuộc "nhảy cóc" – bỏ qua tầng trung gian ở giữa
  - Có thể bố trí một số gói đặc biệt được tham chiếu trong nhiều tầng – VD Utilities

Mẫu kiến trúc phân tầng

- Lý do sử dụng
  - Phân tách các mức độ trừu tượng
    - Các yếu tố có cùng mức đồ trừu tượng được đặt chung trong một tầng
  - Phân tách các mối quan tâm (separation of concerns)
    - Nhóm các yếu tố giống nhau lại
    - Phân tách các yếu tố khác nhau
  - Độ bền bỉ (Resiliency)
    - Ràng buộc lỏng
    - Đóng gói các thay đổi
    - Phân tách các yếu tố thuộc các nhóm khác nhau để có thể dễ thay đổi

Mẫu kiến trúc phân tầng



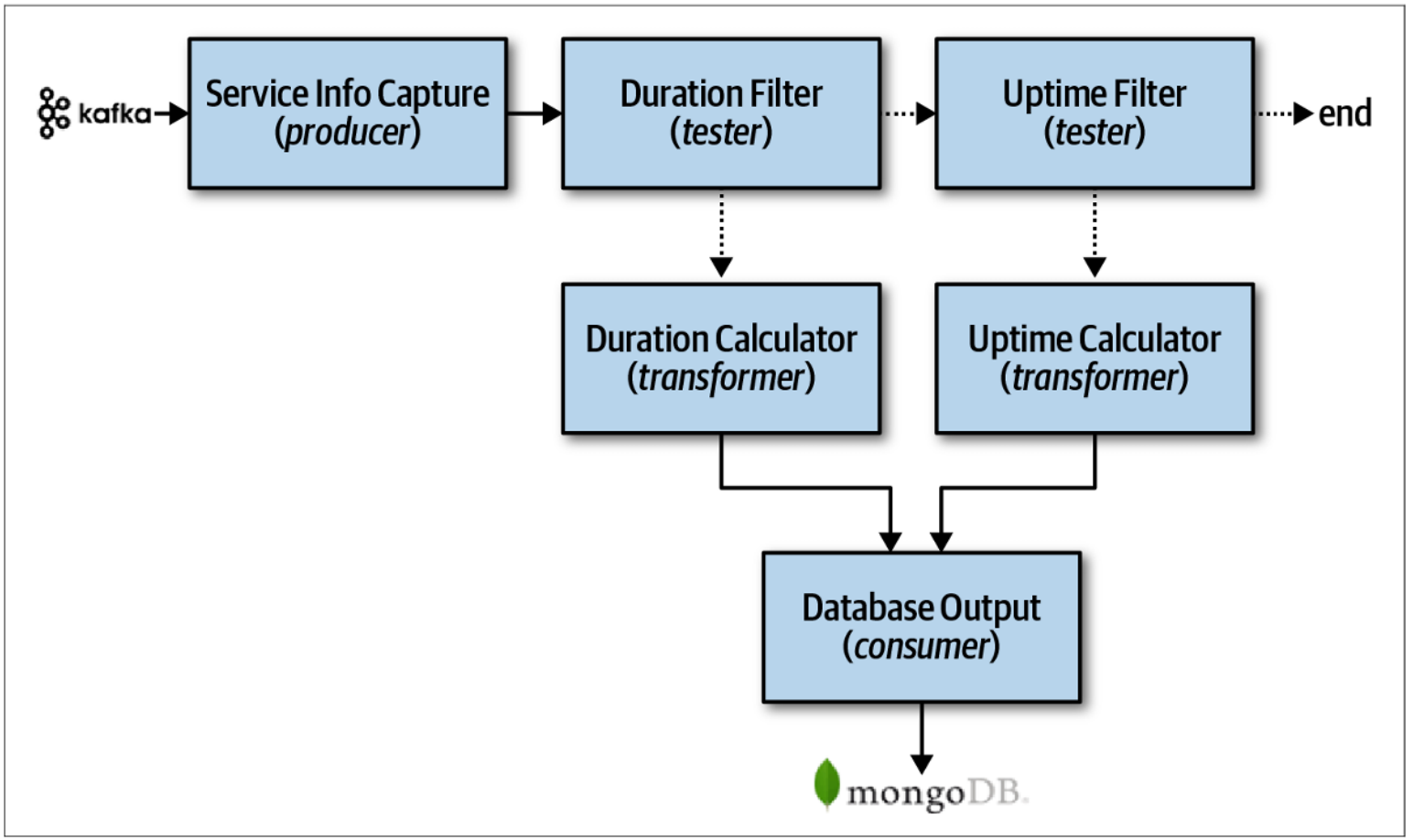
Các topology vật lý khác nhau của kiến trúc phân tầng

Mẫu kiến trúc Pipes & Filters

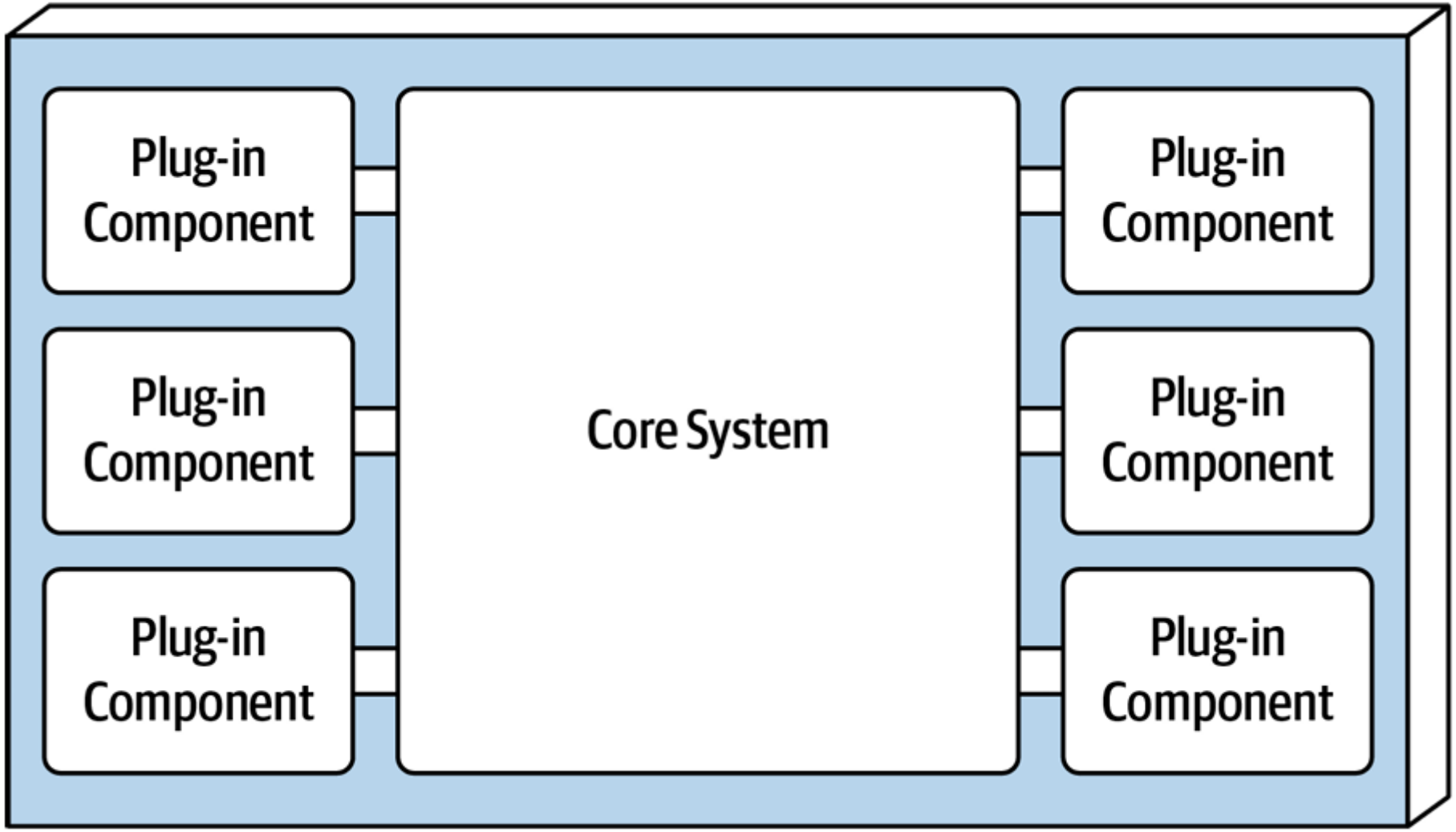


Filter: một module xử lý độc lập, self-contained, thường là stateless
Pipe: mô đun truyền thông (thường là đơn hướng, điểm-điểm) giữa các filter

Mẫu kiến trúc Pipes & Filters – Ví dụ

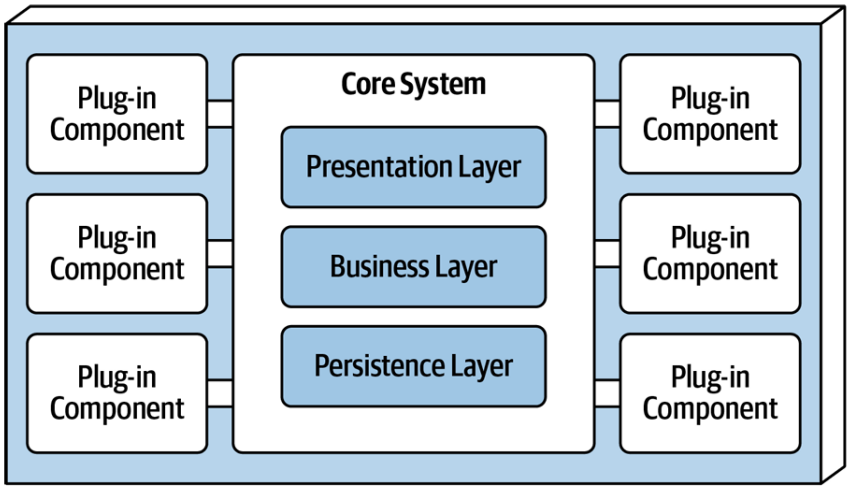


Mẫu kiến trúc Microkernel

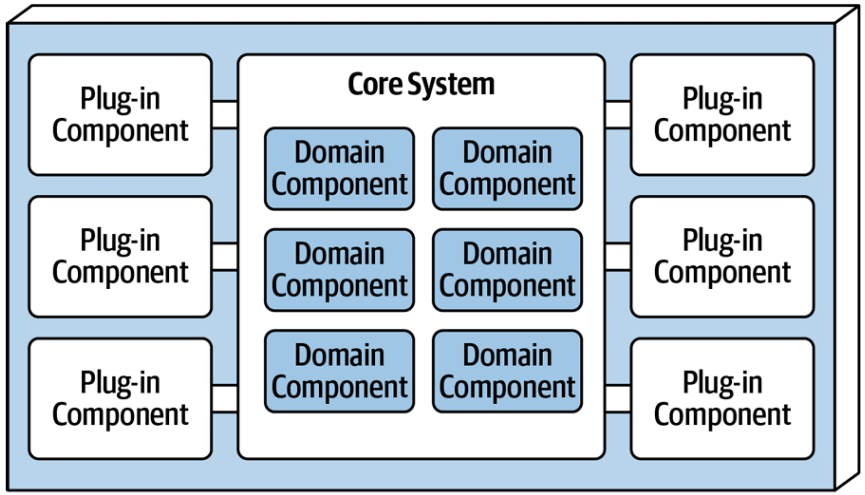


Mẫu kiến trúc Microkernel

- Lõi hệ thống (Core system): Là mô đun cung cấp các chức năng tối thiểu để có thể vận hành hệ thống



Layered Core System (Technically Partioned)

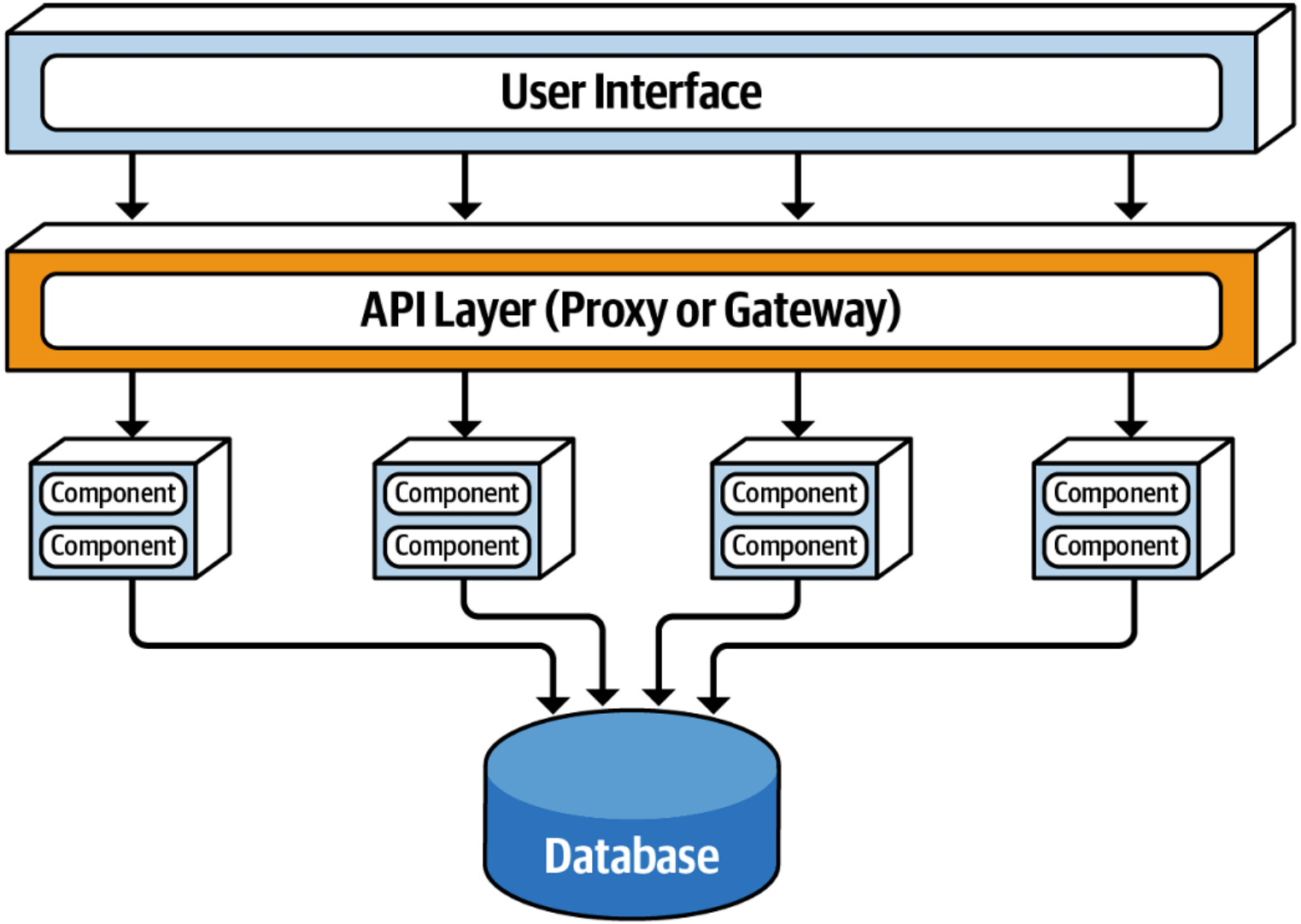


Modular Core System (Domain Partioned)

Mẫu kiến trúc Microkernel

- Plug-in: Là các thành phần độc lập, chịu trách nhiệm cho các công việc xử lý thông tin: mở rộng linh hoạt cho lõi hệ thống.
  - Đăng ký (registry): chứa các thông tin về mỗi plug-in (tên, giao thức,...)
  - Hợp đồng (contract): chuẩn kết nối, giao tiếp giữa plug-in và lõi hệ thống. Rất cần thiết khi plug-in được phát triển bởi bên thứ 3.

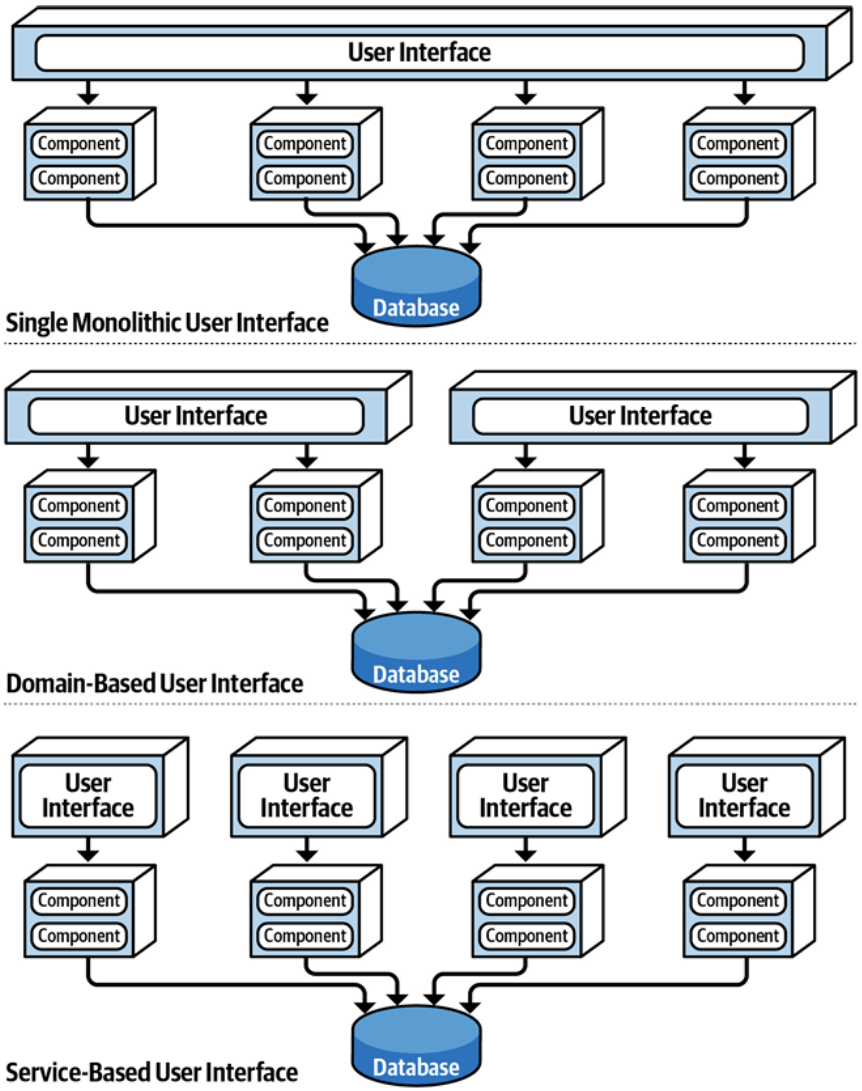
Mẫu kiến trúc dựa trên dịch vụ



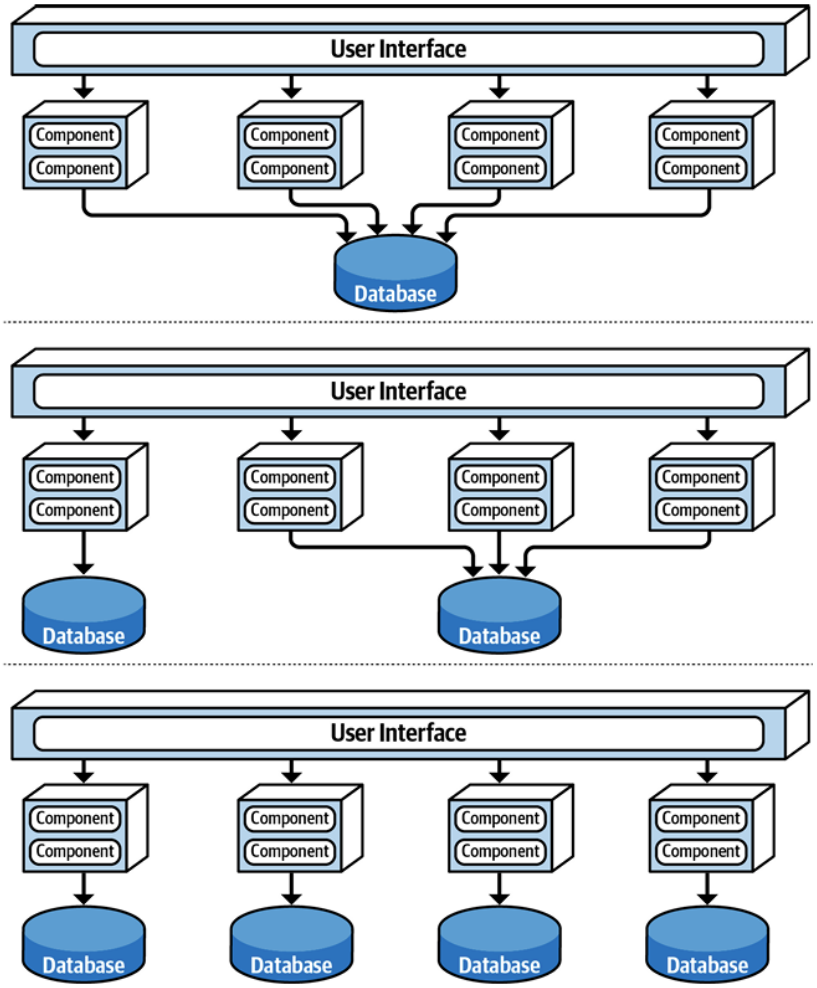
Mẫu kiến trúc dựa trên dịch vụ

- Hệ thống được chia nhỏ thành các đơn vị gọi là microservice
- Các microservice được thiết kế, xây dựng, triển khai độc lập với nhau, tương tác với nhau thông qua Gateway
- Ưu điểm
  - Độc lập trong phát triển
  - Độc lập trong triển khai và tích hợp
  - Kích thước nhỏ, dễ phát triển và kiểm thử
  - Dễ cô lập và thay thế

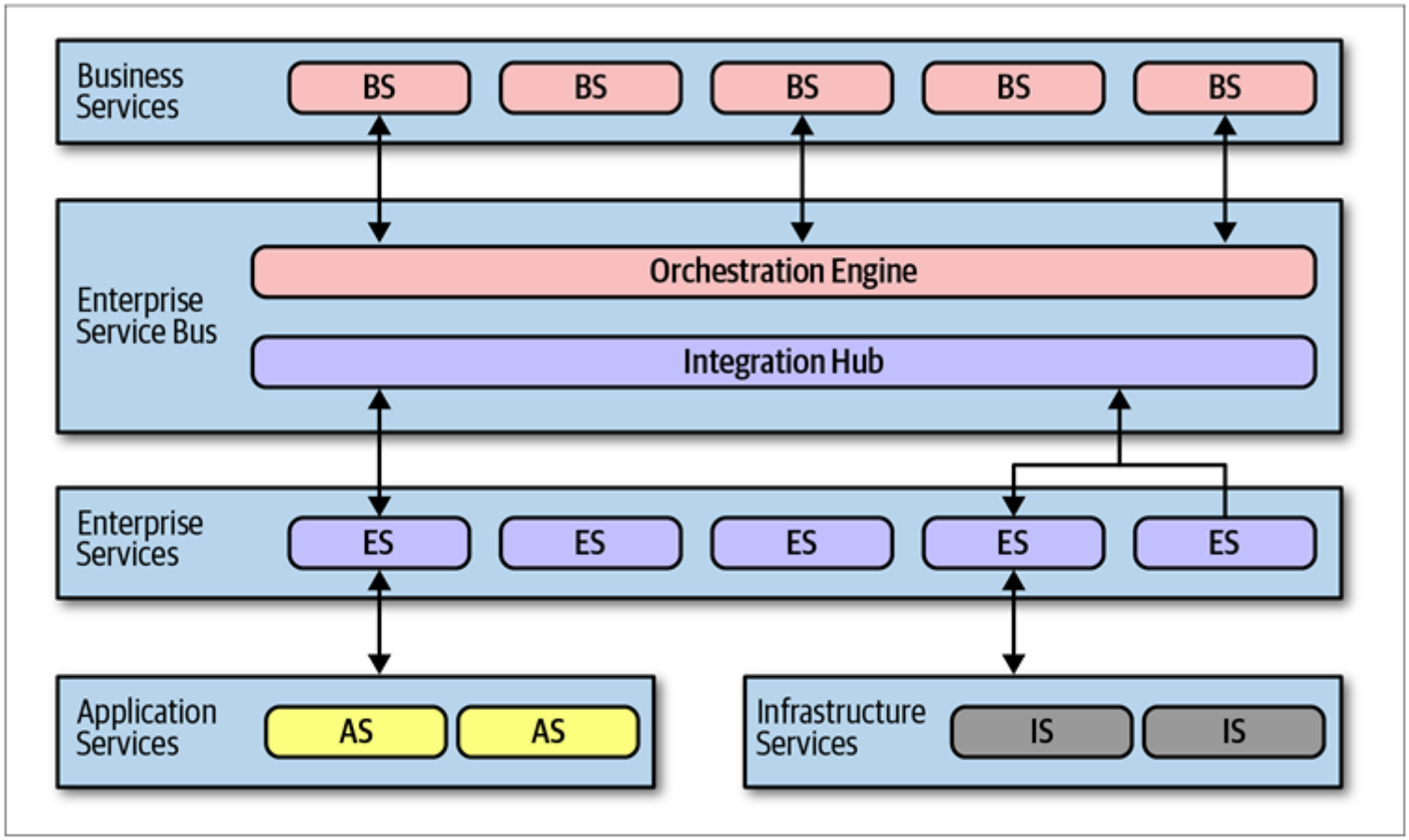
Mẫu kiến trúc dựa trên dịch vụ



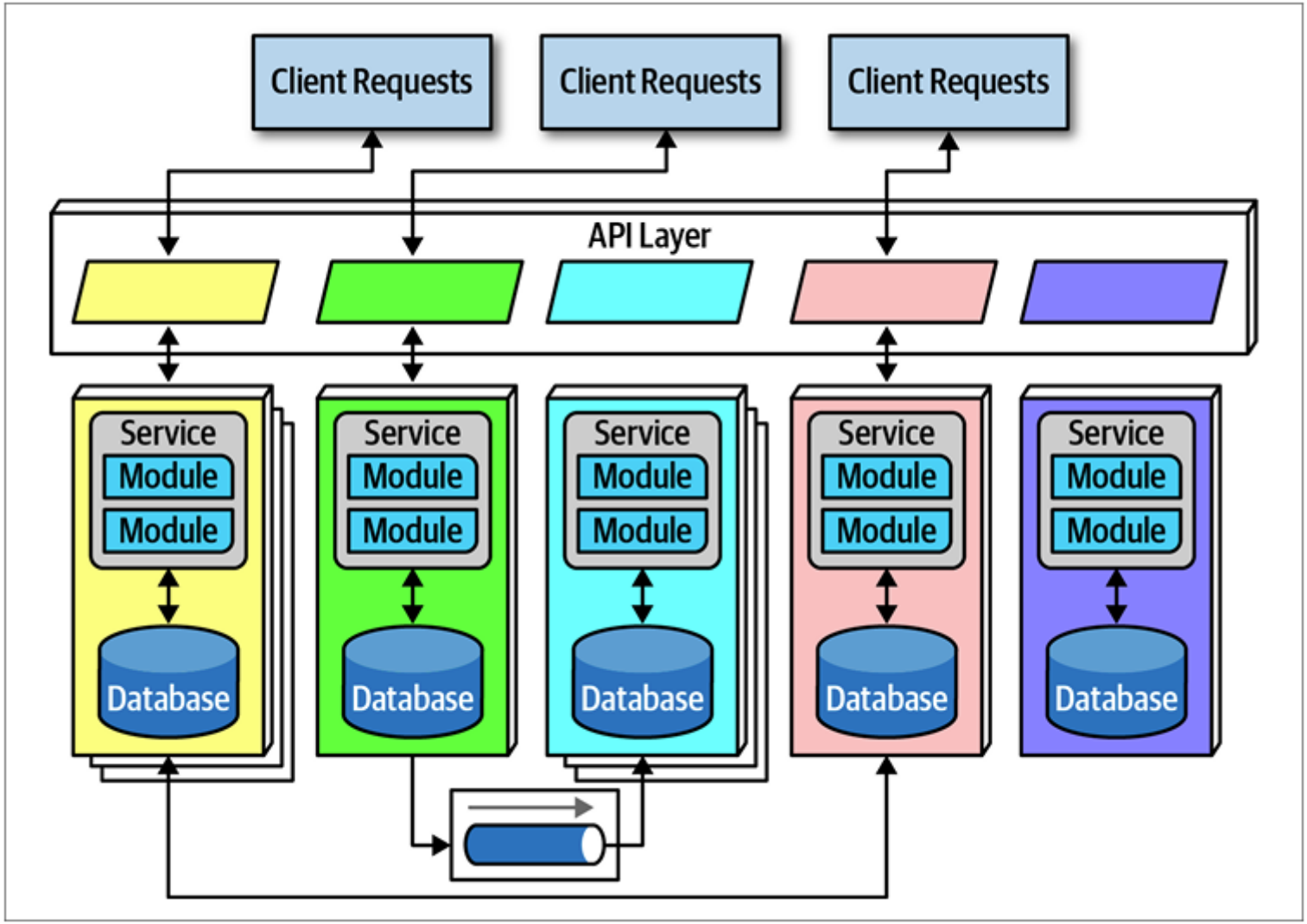
Mẫu kiến trúc dựa trên dịch vụ



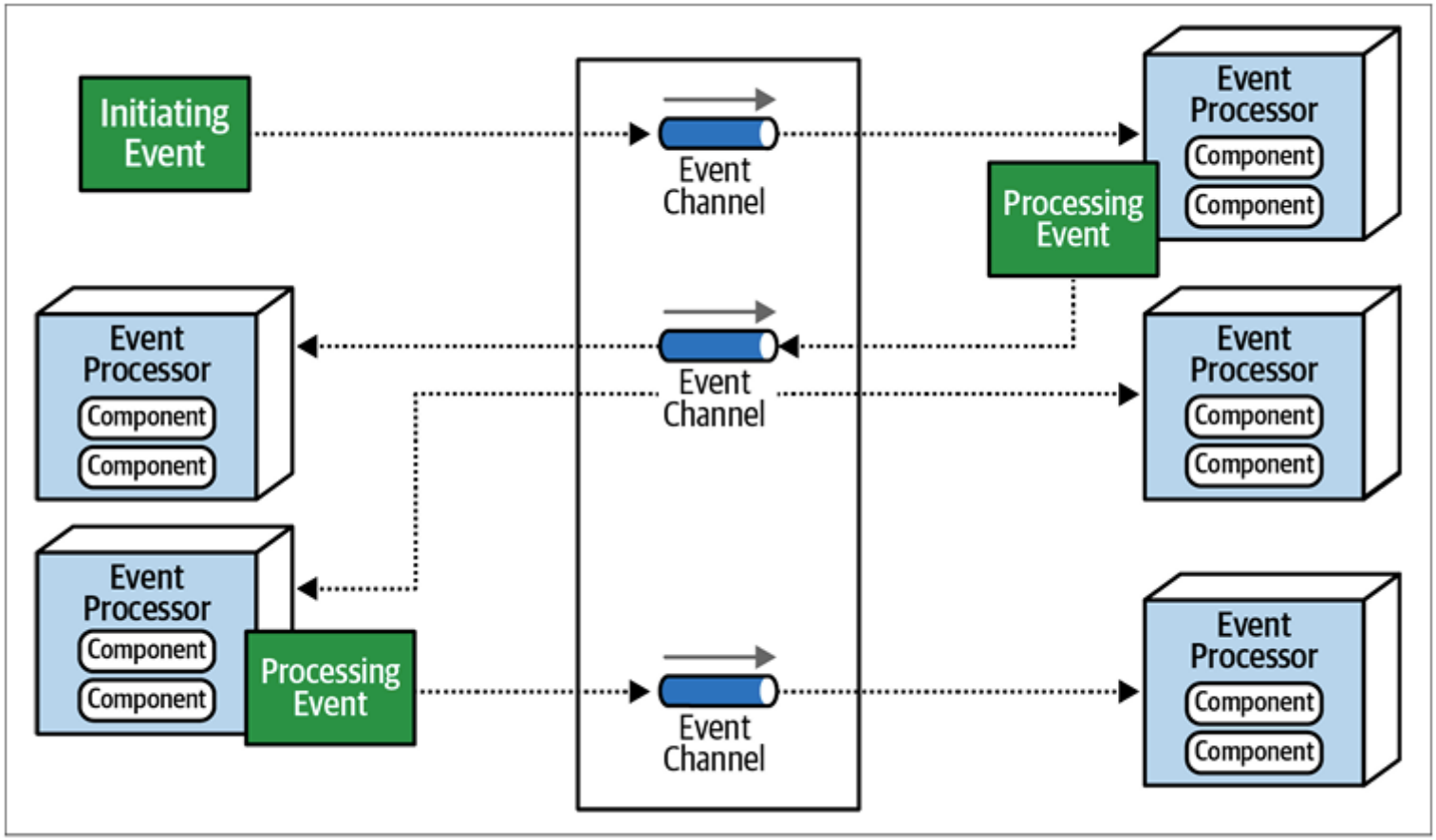
Service-oriented architecture



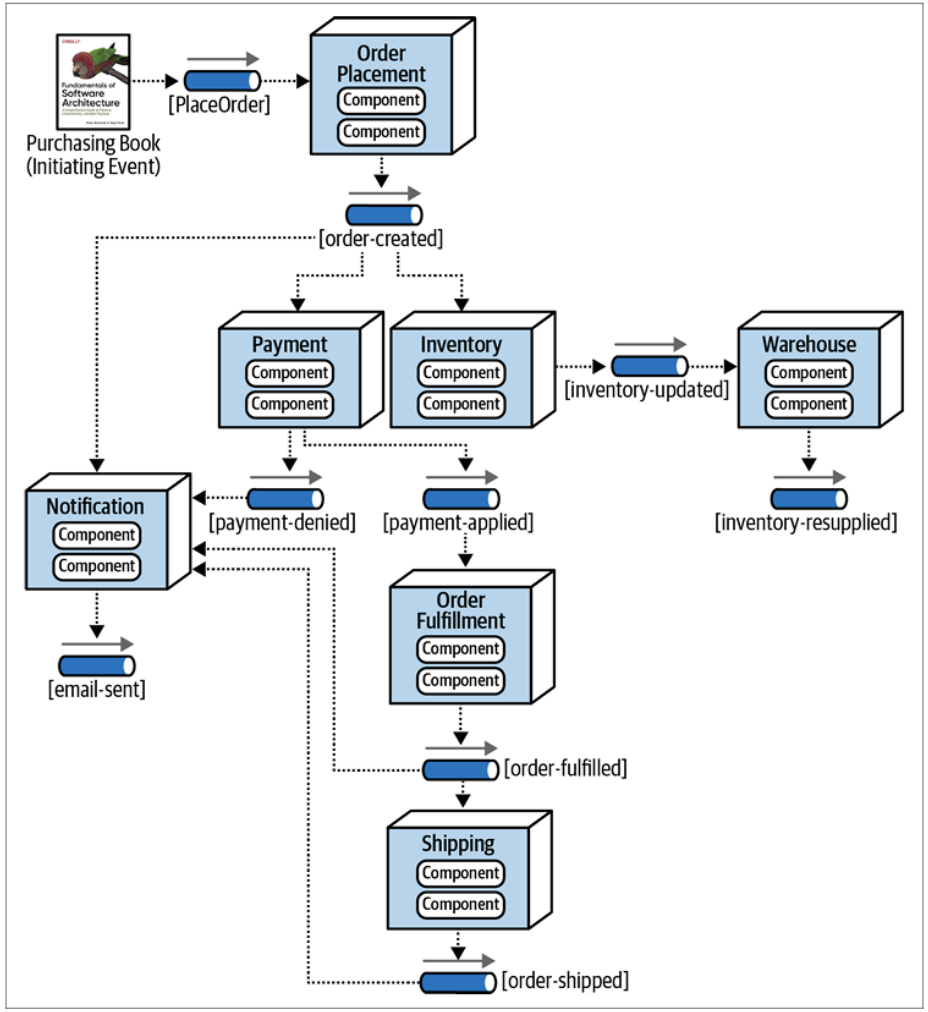
Microservices



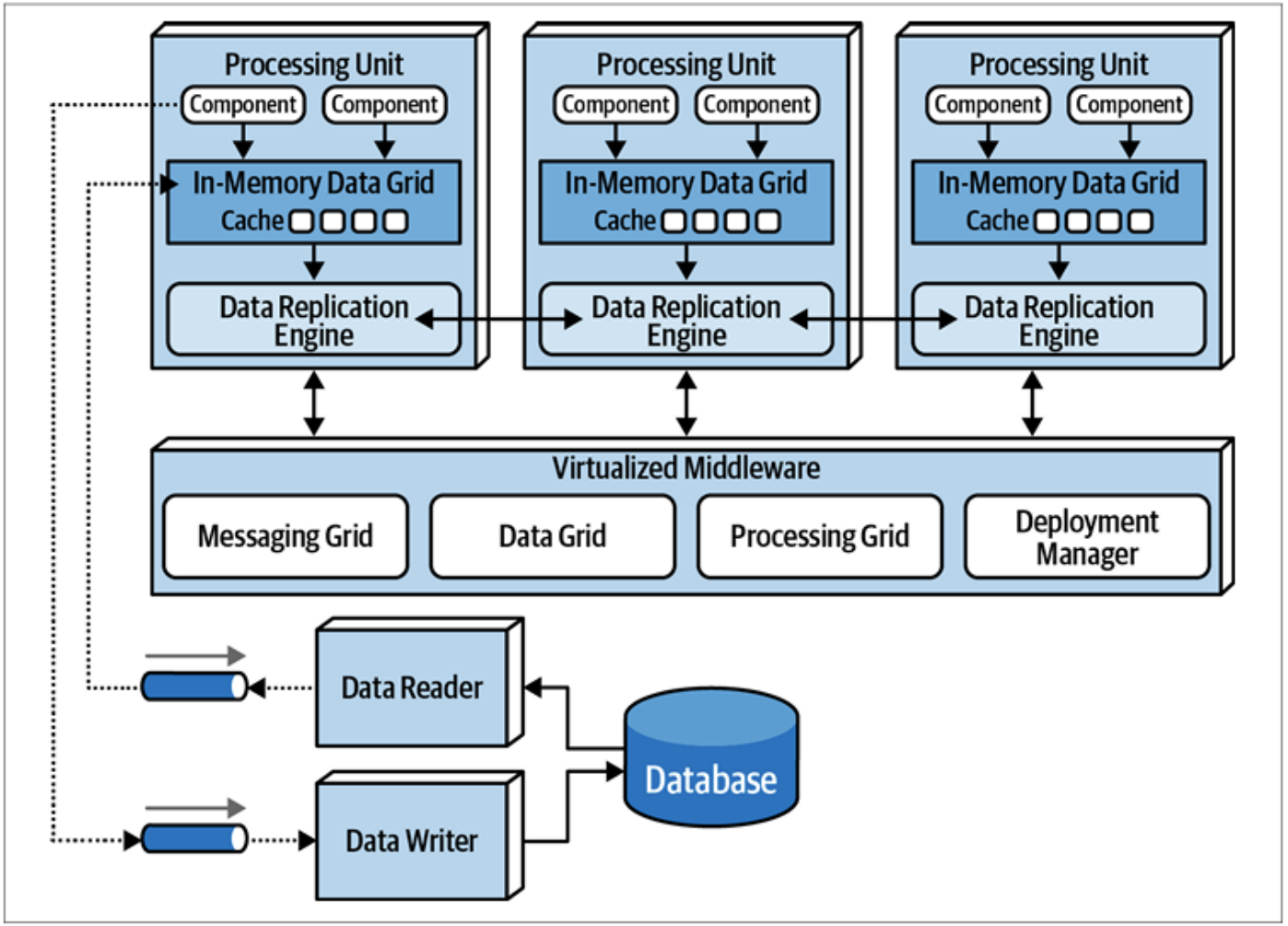
Even-driven architectural style



Even-driven architectural style – Ví dụ

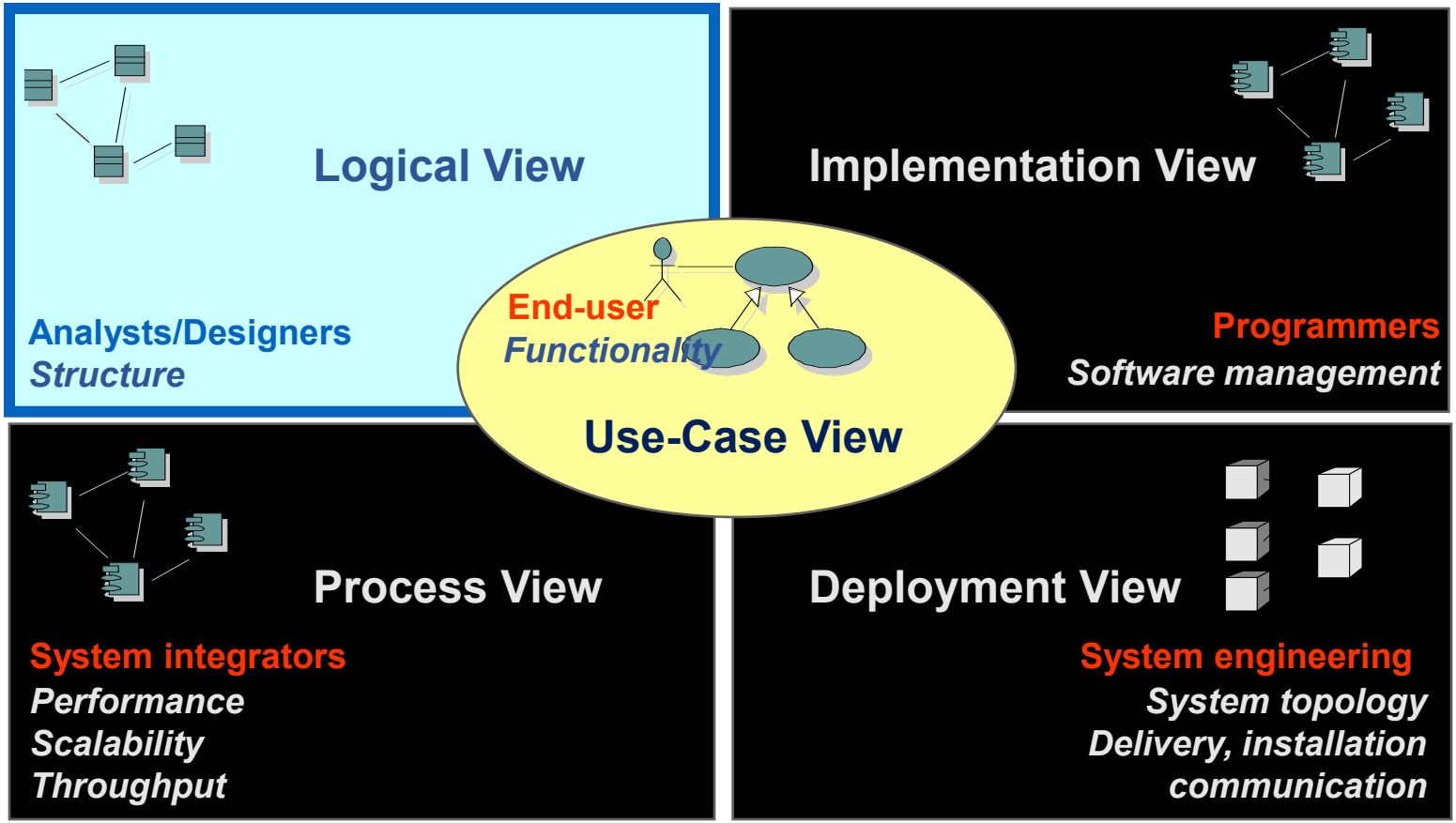


Space-based architectural style



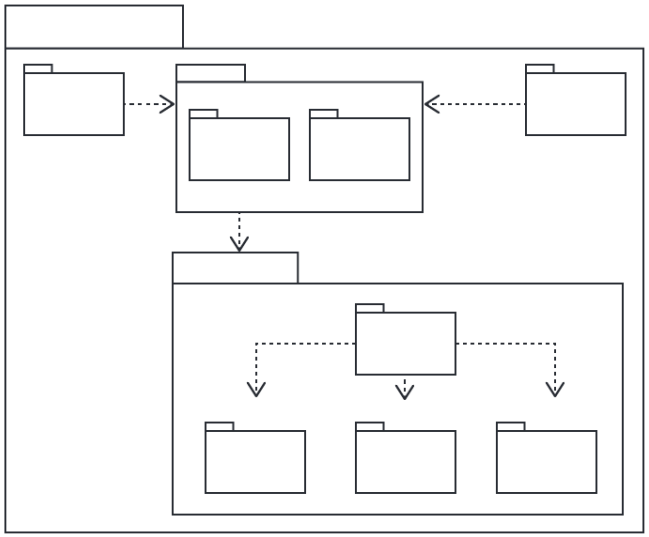
4. Documenting the Architecture in UML

4+1 views


Logical View
Analysts/Designers
Structure
Implementation View
Programmers
Software management
End-user
Functionality
Use-Case View
Process View
System integrators
Performance
Scalability
Throughput
Deployment View
System engineering
System topology
Delivery, installation
communication

Logical View

- Tổ chức các lớp trong hệ thống thành các gói (package) theo nguyên tắc:
  - Cố kết mạnh: liên kết giữa các lớp trong cùng một gói mạnh
  - Tương liên yếu: liên kết giữa các lớp thuộc các gói khác nhau yếu



Sự phụ thuộc giữa các lớp trong gói

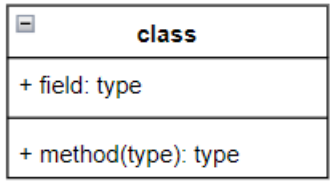
- Cố kết mạnh của các lớp trong cùng một gói thể hiện:
  - **Về mục đích:** Các lớp phải cung cấp các dịch vụ có cùng bản chất cho người dùng. Như vậy chúng phải thuộc vào cùng một lĩnh vực và đề cập một số thuật ngữ chung (chẳng hạn gói giao diện đề cập các thuật ngữ như: cửa sổ, thực đơn, nút nhấn,...)
  - **Về xu thế phát triển:** Người ta tách các lớp bền vững với các lớp có nhiều khả năng thay đổi. Đặc biệt, thường tách các lớp nghiệp vụ với các lớp ứng dụng, và xếp chúng vào các hệ con khác nhau.
  - **Về ứng dụng các công nghệ:** Để tận dụng các công nghệ có sẵn, như các thư viện chương trình (lớp/thành phần), các GUI, các hệ quản trị cơ sở dữ liệu,…, ta thường tách các gói giao tiếp, gói quản trị dữ liệu ra khỏi phần lõi (ứng dụng và nghiệp vụ) của hệ thống

Sự phụ thuộc giữa các gói

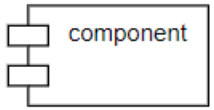
- Tương liên giữa các gói thể hiện ở mối liên quan phụ thuộc giữa chúng
  - Sự phụ thuộc giữa 2 gói phản ánh các mối liên quan tĩnh (thừa kế, liên kết, …) và các mối liên quan động (trao đổi thông điệp) giữa các lớp thuộc 2 gói đó
- Sự phụ thuộc giữa các gói phải càng đơn giản, lỏng lẻo thì càng tốt. Để đảm bảo tính tương liên yếu này, khi thành lập gói, áp dụng các quy tắc sau:
  - Các lớp thuộc vào cùng một cấu trúc thừa kế (inheritance hierarchy) nên được xếp vào cùng một gói
  - Các lớp có mối liên quan kết nhập và hợp thành với nhau thường được xếp vào cùng một gói
  - Các lớp cộng tác với nhau nhiều, trao đổi thông tin nhiều, thể hiện qua các biểu đồ tương tác, thì nên đặt chung vào một gói
  - Nên tránh sự phụ thuộc vòng quanh giữa các gói

Implementation View

- Các lớp được thực hiện thành các thành phần mã nguồn nào?
- Các thành phần mã nguồn được biên dịch thành các thành phần thực thi nào?
- Ứng dụng được đóng gói thành các thành phần bố trí nào



Yếu tố của hệ thống trong góc nhìn logic



Yếu tố của hệ thống trong góc nhìn thực hiện

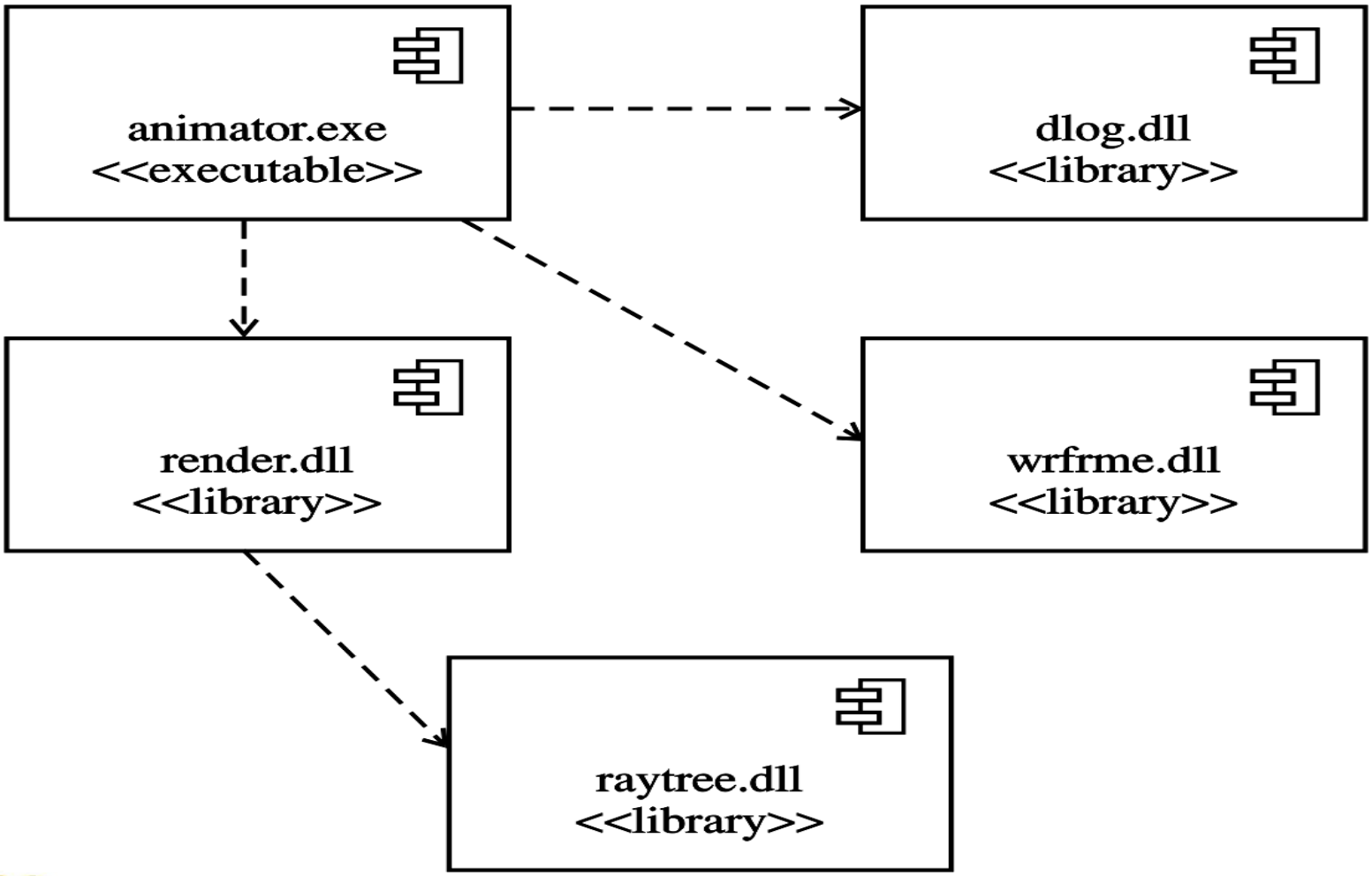
Thành phần (Components) và biểu đồ thành phần (Component diagram)

- Nếu như biểu đồ gói mà ta nói ở phần trên phản ánh cho góc nhìn về cấu trúc logic của hệ thống (ở mức cao so với biểu đồ lớp), thì biểu đồ thành phần cho ta một cách nhìn về cấu trúc vật lý của hệ thống
- Chữ "vật lý" ở đây được hiểu theo nghĩa là sự mô tả hướng tới các sản phẩm phần mềm, là kết quả của sự cài đặt và thực sự tồn tại
  - Chứ không phải là các sản phẩm logic, là kết quả của quá trình phân tích/thiết kế
- Tuy nhiên, ở đây ta chưa đề cập tới phần cứng, mặc dù tính vật lý của nó cũng là đương nhiên

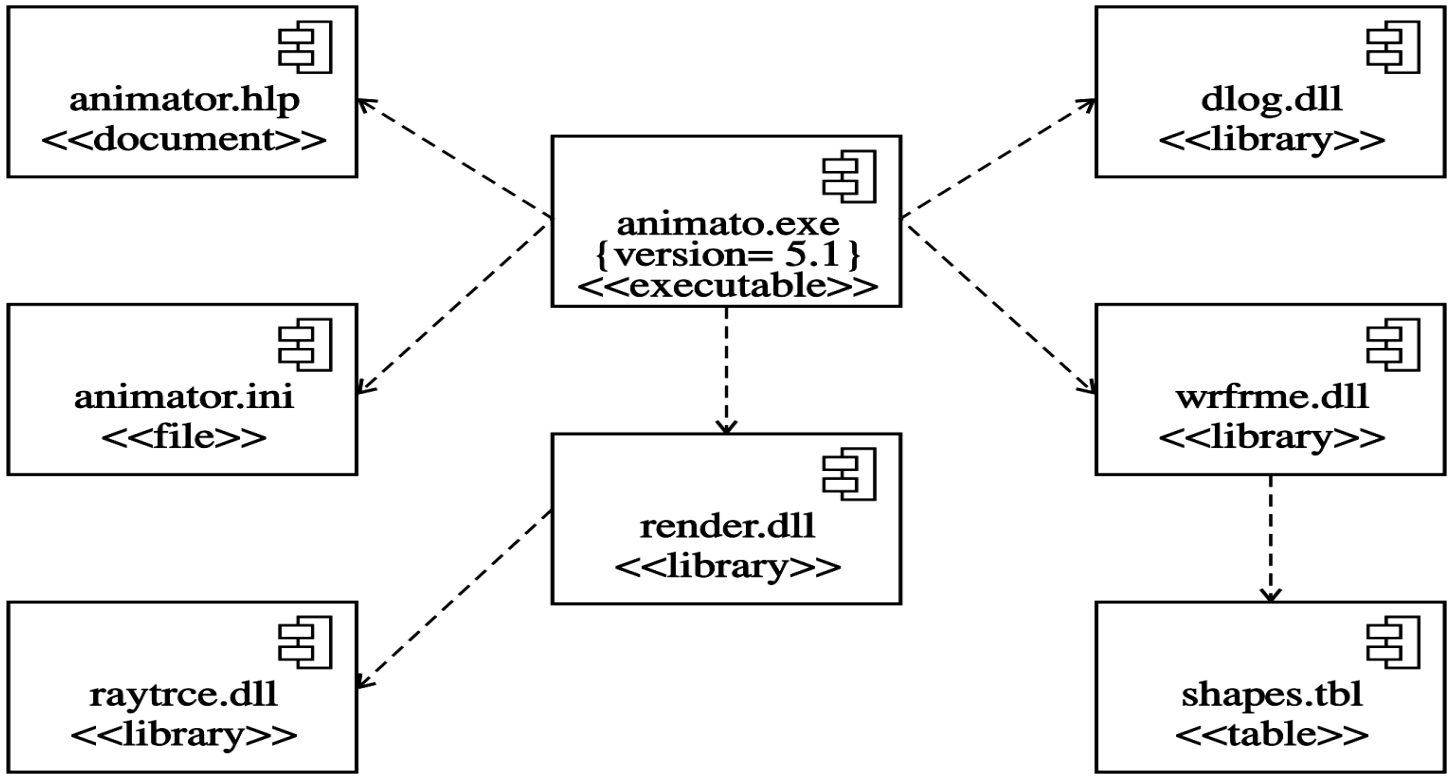
Thành phần (Components)

- UML định nghĩa thành phần (component) là một bộ phận vật lý và thay thế được của hệ thống, thích ứng và cung cấp sự thực hiện (implementation) cho một tập các giao diện (interfaces)
- Nói cách khác, thì thành phần là một cài đặt của một tập hợp các phần tử logic (vd: các lớp hay các hợp tác)

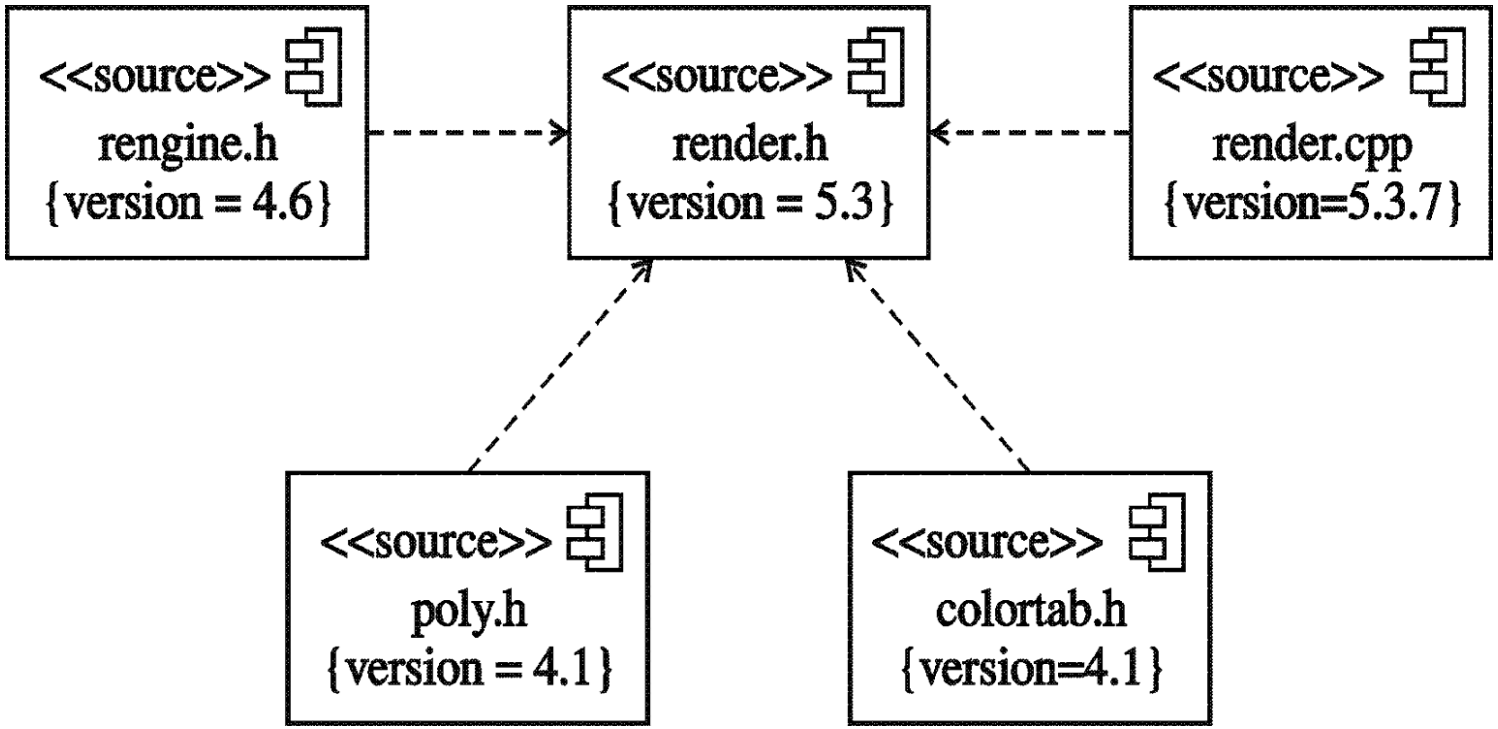
Example 1.


animator.exe
<<executable>>
dlog.dll
<<library>>
render.dll
<<library>>
wrfrme.dll
<<library>>
raytree.dll
<<library>>

Example 2.


animator.hlp
<<document>>
animator.ini
<<file>>
raytrce.dll
<<library>>
animato.exe
{version= 5.1}
<<executable>>
render.dll
<<library>>
dlog.dll
<<library>>
wrfrme.dll
<<library>>
shapes.tbl
<<table>>

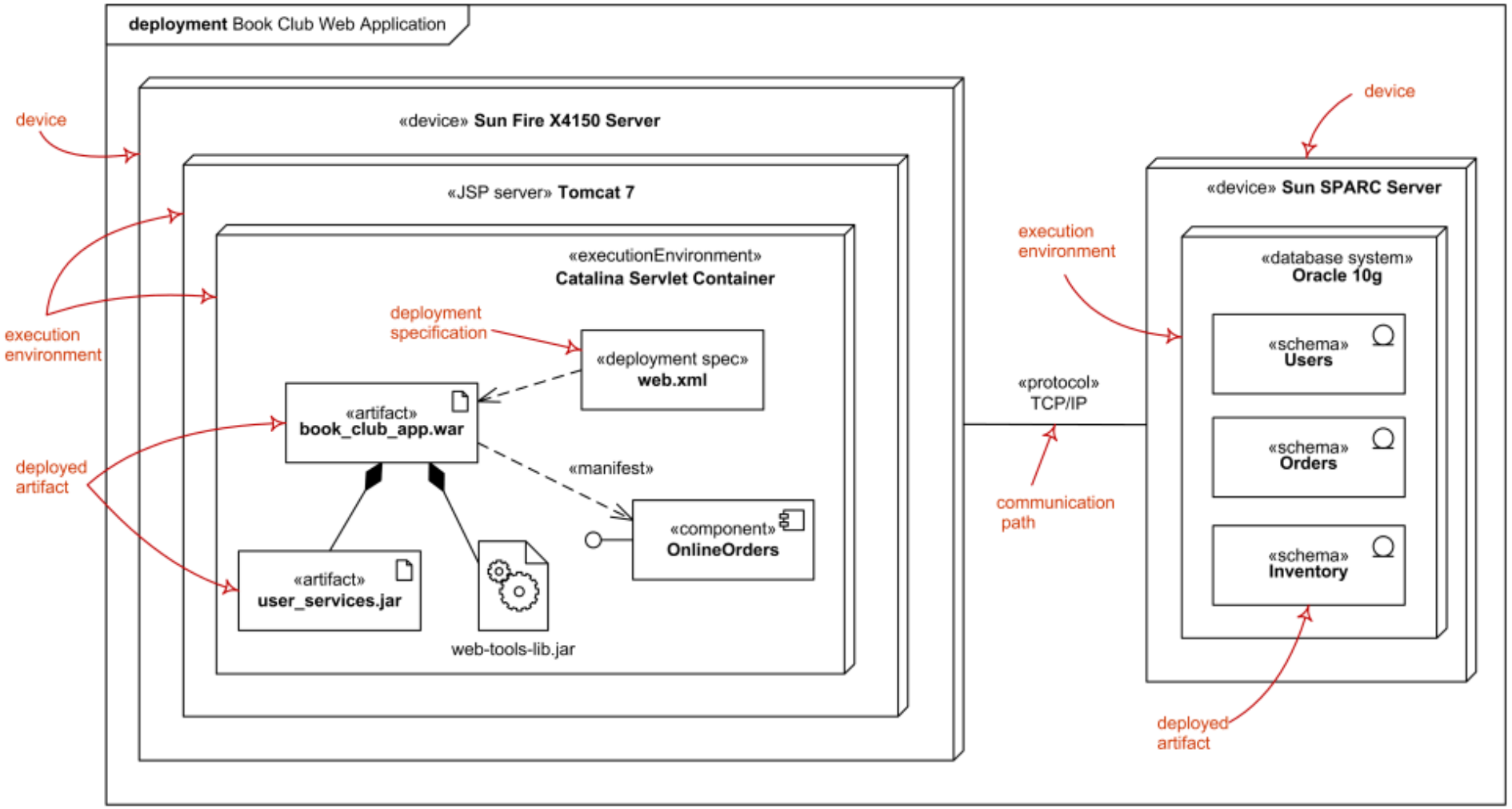
Example 3.


<<source>>
rengine.h
{version = 4.6}
<<source>>
render.h
{version = 5.3}
<<source>>
render.cpp
{version=5.3.7}
<<source>>
poly.h
{version = 4.1}
<<source>>
colortab.h
{version=4.1}

Deployment View

- Các thành phần được bố trí trên các nút phần cứng nào?
- Các nút phần cứng giao tiếp với nhau như thế nào?

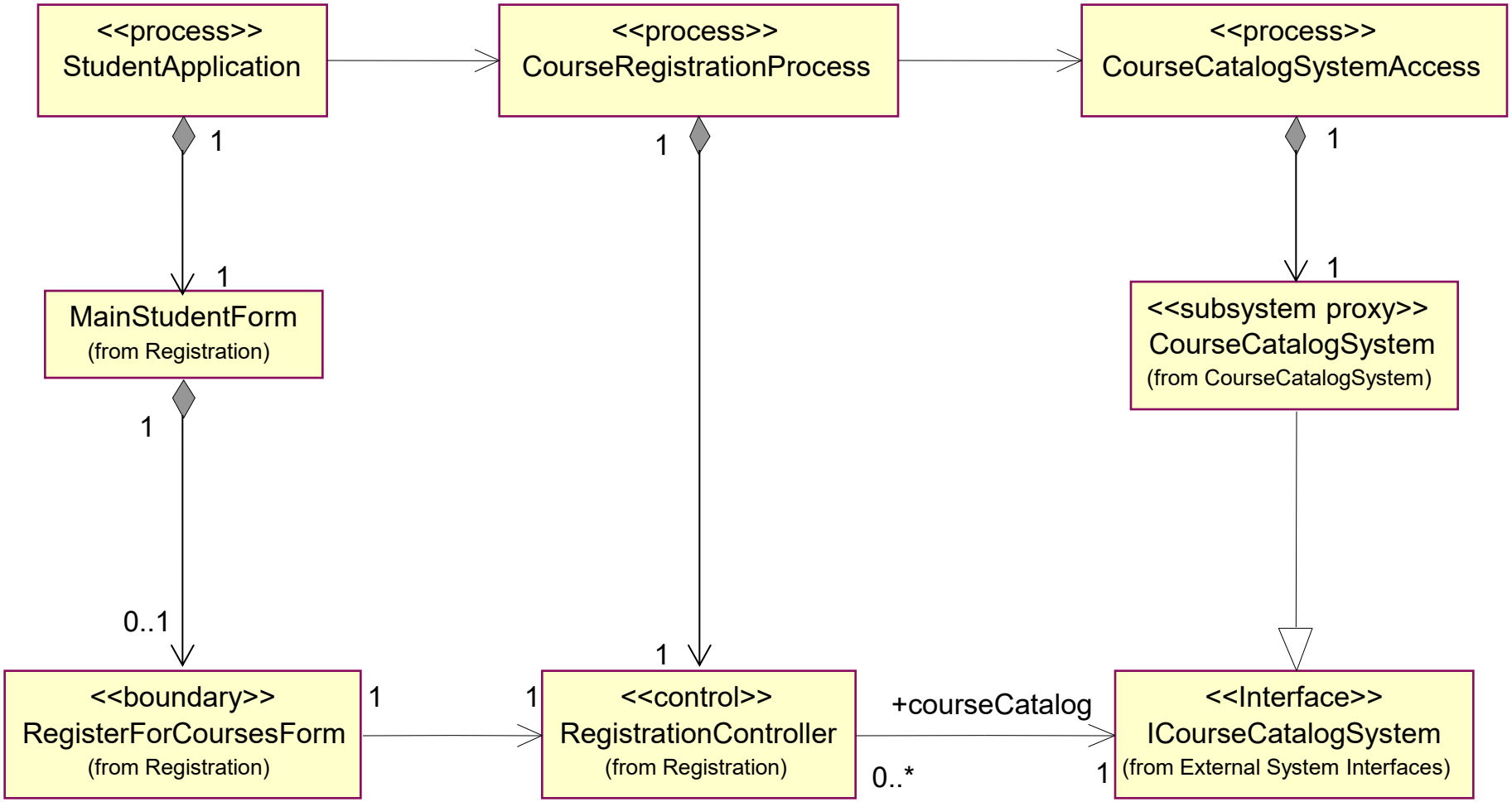
Deployment Diagram



Process View

- Hệ thống được thực thi trên những tiến trình nào?
- Mối quan hệ giữa các tiến trình?
- Các đối tượng của hệ thống được đặt trên các tiến trình nào?

Process View



ĐẠI HỌC BÁCH KHOA

25 YEARS ANNIVERSARY SOICT

VIỆN CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

Chân thành cảm ơn!



soict.hust.edu.vn/ fb.com/groups/soict